



Project name: University System

Discipline: Object-Oriented Programming

Team members:

Frolov Yevgeniy, Student ID: 24B031120

Asset Yeshenbay, Student ID: 24B031026

Kerimkul Kurbanbek, Student ID: 24B031845

Main goal of project:

Main goal of our project to create **Research-Oriented University System**, shortly - “ROUS”. Or even more precisely, imitate behaviour and state of real University system. So, basically, it's creating of management system, which should be convenient for its users and maintainable for its providers. Students and teachers should be able easily access their core actions, while managers and admins should easily control inner side of University management and data. To implement such system we were using java language with newly obtained knowledge of OOP to create simple, scalable, maintainable and adaptable. Our system should support various different objects types and easily handle them, having space for further improvements/optimization.

Project Description:

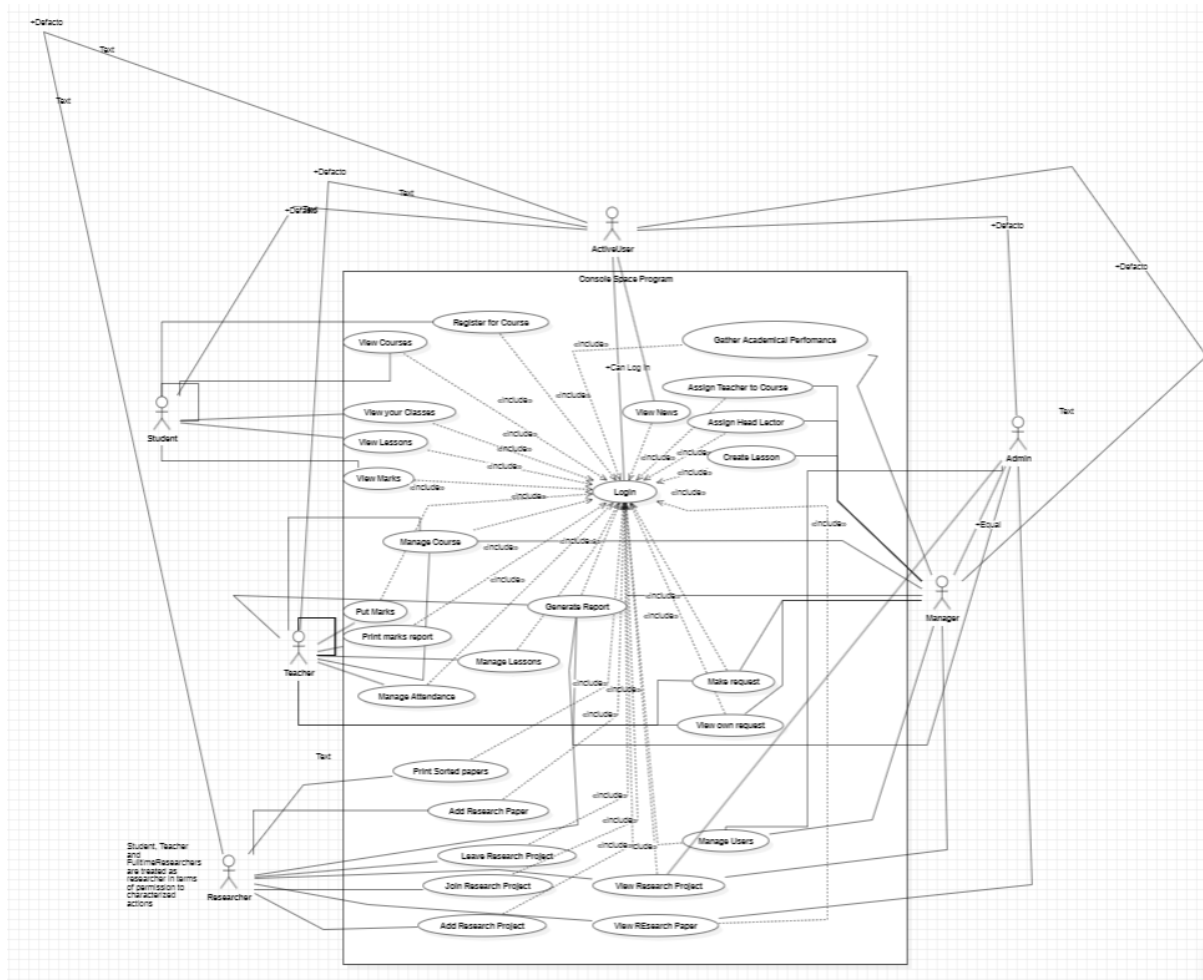
1. Create uml diagrams for: Use case, Objects, Classes(behaviour) before and after refactoring.
2. Write code basing on those diagrams
3. Working with Eclipse, VS Code and Maven to complete the project
4. Make html documentation, presentation, report.

1. Diagrams:

Before coding part we made four UML diagrams using StarUML application and its tools, those are:

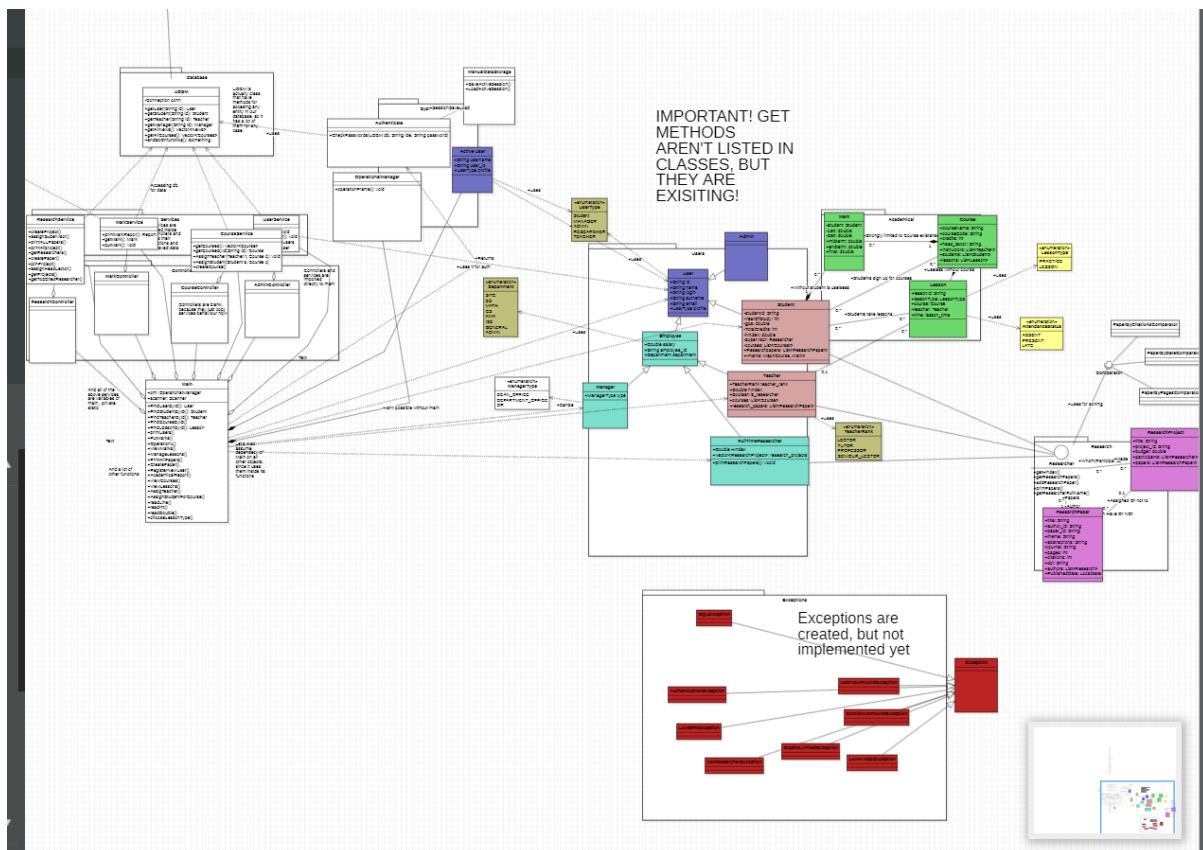
1. Use case diagrams
2. Object diagram
3. Class diagram before refactoring
4. Class diagram after refactoring

1. Use case diagram:



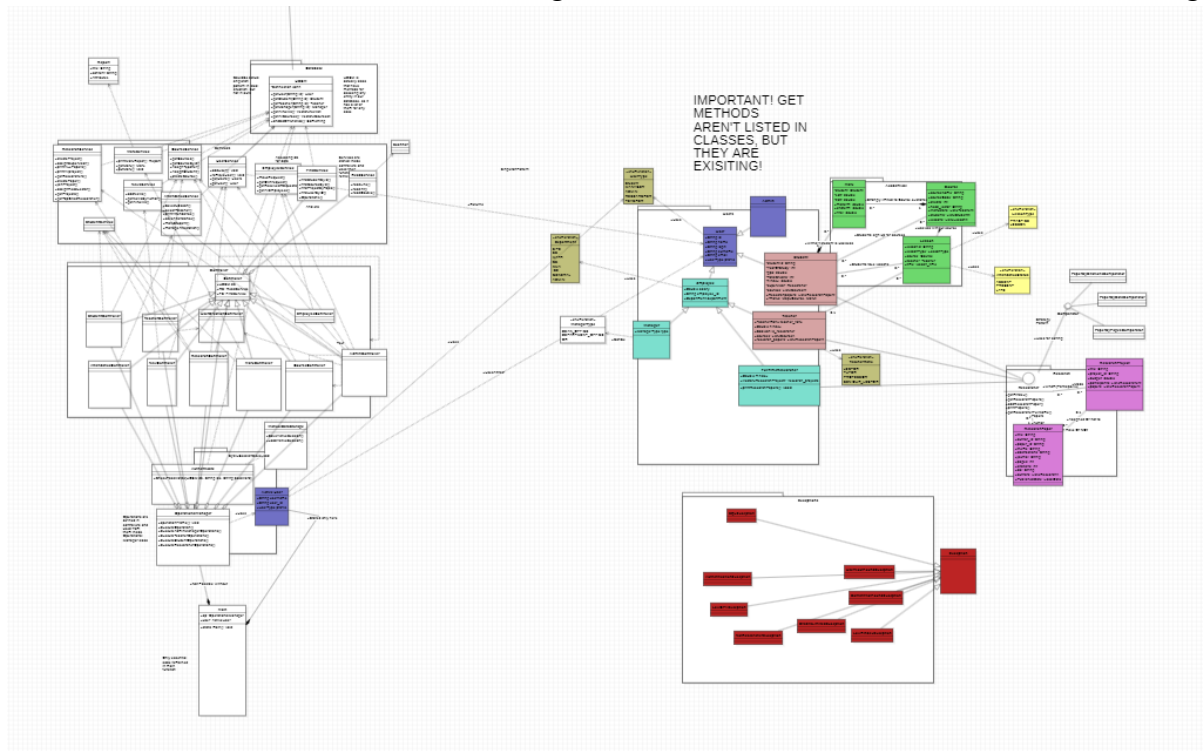
We have 6 actors here: Admin, Researcher, Manager, Student, Teacher and Active User. Active User is actually class needed for interaction with console itself, so every other actor is converted into it during registration and authentication. Also, “Researcher” actor is person who can be teacher, student or pure-researcher. Login into the system is required before any actions.

2. Object Diagram:



We were trying to implement system: Database -> Services -> function controllers -> Console. We haven't done it fully yet at that stage and met obstacles. We were planning to distribute heavy augmented main content among controllers rightfully.

4. Class diagram after refactoring:



To fix previous troubles we made more controllers to make code more maintainable, we made

Operational Manager as singleton, fully implemented services functional, separated database from other classes excepting services(mostly). Strategy patter is applied for comparators. Also we implemented choice of function/commands inside Operational manager, to make work faster and code more visible/maintainable.

2. Writing Basic Code:

academic	Add files via upload
comparators	Add files via upload
controllers	added news feature
database	added teacher-student view feature with user info a
demo	added teacher-student view feature with user info a
enums	Add files via upload
exceptions	Add files via upload
news	added news feature
oop	added news feature
reports	Add files via upload
research	Add files via upload
services	added news feature
storage	Add files via upload
system	added teacher-student view feature with user info a
users	added teacher-student view feature with user info a

We can't actually show what was obtained after basic class coding, because we already changed it and have no backups, but we will still tell what there were: Initial classes with lack of methods and non-sufficient fields, wrong architecture, no connection to db at all, poor controller usage. We started improving those problems at next stage of the project.

3. Working with IDEs(VS Code with Maven, Eclipse)

We divided our project into 14 packages, such as **users**, **services**, **controllers**, and others, to organize our classes effectively.

Users – contains user-derived classes and profiles such as Teacher, Manager, and others.

System - package which is containing classes needed for our demo working state. Operational Manager, Authentication and Active User are located here.

Demo – package where is our “Main.java” class is located

Academic – contain Course, Mark and Lesson classes.

Enums – contains enums such as ManagerType, UserType, and others. These enums help us write conditional logic and separate permissions more effectively..

Database - contains a single class, UDBM, which is responsible for working with our database.

Storage - contains classes for file streams, authorization, and session saving, more precisely: ManualDataStorage and DataStorage.

In the Student class, we implemented the required, for our task, fields:

```
public class Student extends User implements Researcher{
    private String studentId;
    private int yearOfStudy;
    private double gpa;
    public boolean is_researcher;
    private int totalCredits;
    private double hindex;
    private List<Course> courses;
    private Map<Course, Mark> marks;
    private Researcher supervisor;
    private List<ResearchPaper> researchPapers;
```

They all get appropriate number of getter and adding methods, but are lacking setters methods. Functional logic of Student is stored inside StudentController class and Operational Manager class:

```
public void printYourClasses(Active user, Scanner scanner){
    Student s = SSE.getStudentById(this.db, user.user_id);
    Vector<Course> cs = CS.getCoursesByStudent(s);
    for (Course c : cs)
    {
        System.out.println(c);
    }
}

public void printLessons(Active user, Scanner scanner){
    Student s = db.getStudentById(user.user_id);
    Vector<Course> cs = CS.getCoursesByStudent(s);
    for (Course c : cs)
    {
        System.out.println(c);
    }
    String c_id = RS.readLine(message: "Select Course", scanner);
    Course cou = new Course();
    try {
        cou = CS.getCourseById(c_id);
        Vector<Lesson> ls = db.getStudentLessonsByCourse(c, cou);
```

```

private boolean executeStudentOperation(int choice, Active user, Scanner scanner) throws Exception {
    switch (choice) {
        case 2:
            CC.printAllCourses();
            break;

        case 3:
            SC.signUpForCourse(user, scanner);
            break;

        case 4:
            MK.printMarks(user, scanner);
            break;

        case 5:
            SC.printYourClasses(user, scanner);
            break;

        case 6:
            SC.printLessons(user, scanner);
            break;
    }
}

```

- and other functions. We implemented functions to view marks, classes, lessons, own projects and own papers, to sign up for course and view news. All of those methods are implemented inside controllers. Some controllers share functions for different user types. For example MarkController contains methods for Teacher and Student. Sometimes those controllers use function overloading to differ methods for different classes :

```

public void putMark(Scanner scanner) throws Exception {
    Student student = FS.findStudentById(scanner);
    Course course = FS.findCourseByCode(scanner);

    double midterm = RS.readDouble("Midterm", scanner);
    double endterm = RS.readDouble("Endterm", scanner);
    double att1 = RS.readDouble("First attestation: ", scanner);
    double att2 = RS.readDouble("Second attestation: ", scanner);
    double finalExam = RS.readDouble("Final exam: ", scanner);

    MS.putMark(student, course, midterm, endterm, att1, att2, finalExam);
    System.out.println("Mark added.");
}

public Report generateReport(Course course) {
    return MS.generateMarksReport(course);
}

public void printMarksReport(Scanner scanner) throws Exception {
    Course course = FS.findCourseByCode(scanner);
    Report report = generateReport(course);
    report.print();
}

```



```

public void printMarks(Active user, Scanner scanner){

    Student s = SSE.getStudentById(db,user.user_id);
    Vector <Course> cs = db.getStudentsCourses(s);
    for (Course c : cs){
        Vector <Mark> marks = db.getMarksByCourse(c);
        for (Mark m : marks){
            if (m.getStudent().getId().equals(user.user_id)){
                System.out.println(m);
            }
        }
    }
}

```

Also we implemented a wide method variety for our admins and managers. Full list of their commands:

```

u:
Please, select the command
1. Exit
2. Register new manager
3. Register new admin
4. Register new teacher
5. Register new student
6. Register new researcher
7. Register new course
8. Assign head lector
9. Assign teacher to course
10. Register student to course
11. Create lesson
12. Assign supervisor
13. Print all users
14. Print all courses
15. Print all papers
16. Print top cited researcher
17. Create research project
18. Make request
19. Get own requests
20. Get received requests
21. Add news
22. Print all employees
23. Set attendance
24. View news
25. View news by author
0. Close console
Enter your command:

```

Important notice, we combined manager and admin functions into one permission level.

```

Fields                                of                                Employee:
private String employeeId;
    private double salary;
    private DepartmentType department;

    public Employee(String id, String email, String login, String name, String
surname,
                    String role, String employeeId, double salary, String
department) {
    super(id, email, login, name, surname, role);
    this.employeeId = employeeId;
    this.salary = salary;
    this.department = DepartmentType.valueOf(department);
}

```

Manager has additional managerType field, which linked to corresponding enumeration.
Admin has no additional fields and methods at all:

```

public class Admin extends Employee {
    public Admin(String id, String email, String login, String name, String
surname,
                String employeeId, double salary, String department) {
        super(id, email, login, name, surname, "ADMIN", employeeId, salary,
department);
    }
}

```

Researcher shares functions with students and teachers, but students and teachers who are not “research_active” won’t be able to conduct these functions, special type of exception will be thrown. Also we made exception for handling SQL problems and inner logic, as example ContentNotFoundException or LowGPAException and etc. :

```

public void assignSupervisor(Student student, Researcher supervisor) throws
LowHIndexException \ \ These aren't parts of same code, just examples combined

public void joinProject(ResearchProject project, User user) throws
NotResearcherException {
    if (user.getProfile() != UserType.STUDENT && user.getProfile() !=
UserType.RESEARCHER && user.getProfile() != UserType.TEACHER)
    {
        throw new NotResearcherException("Only researchers can join");
    }
    if (user.getProfile() == UserType.STUDENT)

```

```

    {
        Student st = db.getStudentByID(user.getId());
        if (!st.is_researcher && st.getGpa() > 3.5 && st.getYearOfStudy()
>=4){
            db.updateStudent(st.getId(), st.getYearOfStudy(), st.getGpa(),
3, true, db.getAllResearchers().elementAt(0).getId());
            db.addResearcher(st.getId(), 3);
        }
        if (st.getGpa() <3.5 || st.getYearOfStudy() < 4){
            throw new NotResearcherException("Not enough score");
        }
    }

```

Teachers have such commands:

```

Welcome! Press x to stop
Login: teacher_arman
Password: pass123
Login successful!

Hello, teacher_arman Welcome to University system, please, choose an action availab
le for you:
Please, select the command
1. Exit
2. Put mark
3. Print marks report
4. Print all courses
5. Print marks for course
6. Set attendance
7. Become Researcher
8. Join research project
9. Start research project
10. Create research paper
11. Make request
12. Get own requests
13. Get received requests
14. View news
15. View news by author
0. Close console
Enter your command: █

```

These functions were made according to already existing plan using controllers, services and etc. Teacher, Student, FullTimeResearcher and Manager are more like profiles and permissions, but they still can be built as class instance in some cases. For example, when we need to print them. Also, we granted Employee full class rights for the same reason:

```

public class Employee extends User {
    private String employeeId;
    private double salary;
    private DepartmentType department;

    public Employee(String id, String email, String login, String name, String
surname,

```

```

        String role, String employeeId, double salary, String
department) {
    super(id, email, login, name, surname, role);
    this.employeeId = employeeId;
    this.salary = salary;
    this.department = DepartmentType.valueOf(department);
}
// -----Below is function from
AdminController
public void printAllEmployees(){
    Vector <Employee> emps = EC.getAllEmployee(db);
    for (Employee em : emps){
        System.out.println(em);
    }
}

```

Academical report is implemented just for marks:

```

public Report generateMarksReport(Course course) {
    double sum = 0;
    int count = 0;

    for (Mark marke : db.getMarksByCourse(course)) {
        if (marke != null) {
            sum += marke.getTotal();
            count++;
        }
    }

    double average = count == 0 ? 0 : sum / count;
    return new Report("Marks report for " + course.getCourseName(),
        "Students with marks: " + count + "\nAverage total mark: " +
average);
}

```

Where Report is separate class:

```

package reports;

import java.io.Serializable;

public class Report implements Serializable {
    private String title;
    private String content;

    public Report(String title, String content) {
        this.title = title;
    }
}

```

```

        this.content = content;
    }

    public void print() {
        System.out.println("==== " + title + " =====");
        System.out.println(content);
    }

    @Override
    public String toString() {
        return title + "\n" + content;
    }
}

```

Also, another important thing about our project, researcher functions are implemented by
 Researcher interface:

```

package research;

import java.util.Comparator;
import java.util.List;

public interface Researcher {
    double getHIndex();
    List<ResearchPaper> getResearchPapers();
    void addResearchPaper(ResearchPaper paper);
    void printPapers(Comparator<ResearchPaper> comparator);
    String getResearcherFullName();
}

```

So, we manually add fields such as hIndex or List/Vector<ResearchProject> to our interface implementing classes: FullTimeResearcher, Student and Teacher.

Also we implementer “Serializable” interface for our classes, to quickly make local data savings via fileStreams, if there is a case(We might not need database in every app, maybe).

And our Main.java:

```

package demo;

import java.util.Scanner;

import database.UDBM;
import services.ReadService;
import storage.ManualDataStorage;
import system.Active;
import system.Authenticate;
import system.OperationalManager;

```

```

public class Main {
    private static final Scanner scanner = new Scanner(System.in);

    private static final UDBM db = new UDBM();
    private static final ReadService RS = ReadService.getInstance();
    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println(" Research-Oriented University System");
        System.out.println(" Console Demo");
        System.out.println("=====");

        OperationalManager op = OperationalManager.getInstance(db);
        boolean loginer = Authenticate.consolewin(db, scanner);

        while (op.running) {

            if (loginer){
                Active actuser = ManualDataStorage.loadSession();
                op.setType(actuser.profile);
                System.out.println("Hello, " + actuser.username + " Welcome to
University system, please, choose an action available for you: ");
                op.operationFrame();
                int choice = RS.readInt("Enter your command: ", scanner);
                if (choice != 1){
                    try {
                        op.executeOperation(choice, actuser.profile, actuser,
scanner);
                    }
                    catch (Exception e) {
                        System.out.println(e.getMessage()); //TODO, Make More
exceptions
                    }
                }
                else{
                    loginer = false;
                }
            }

            else{
                loginer = Authenticate.consolewin(db, scanner);
            }

            System.out.println();

```

```

    }
}
}

```

Login function from Authenticate returns boolean values, so we can use it and separate variable (loginer) for controlling different users session. Operation manager receives permission type, calls command printing and then receives integer for its switch case. Reading from keyboard is implemented in separate service “ReadService”, searching some main instance in DB is implemented in “FindService”:

```

package services;

import java.sql.SQLException;
import java.util.Scanner;
import java.util.Vector;

import academic.Course;
import database.UDBM;
import exceptions.ContentNotFoundException;
import exceptions.UserNotFoundException;
import research.ResearchProject;
import research.Researcher;
import users.Student;
import users.Teacher;
import users.User;

public class FindService {
    private final ReadService rs = ReadService.getInstance();
    UDBM db;
    public FindService(UDBM db){
        this.db = db;
    }

    public static FindService getInstance(UDBM db){
        return new FindService(db);
    }

    public Course findCourseByCode(Scanner scanner) throws
ContentNotFoundException {
        String code = rs.readLine("Course code: ", scanner);

        try {for (Course course : db.getAllCourses()) {
            if (course.getCourseCode().equalsIgnoreCase(code)) {
                return course;
            }
        }
    }
}

```

```
    }  
    }  
}\\And so on
```

```
package services;  
  
import java.util.Scanner;  
  
public class ReadService {  
    public ReadService(){  
  
    }  
  
    public static ReadService getInstance(){  
        return new ReadService();  
    }  
    public String readLine(String message, Scanner scanner) {  
        System.out.print(message);  
        return scanner.nextLine();  
    } // and so on
```

4. Documentation

Packages	
Package	Description
academic	
comparators	
controllers	
database	
demo	
enums	
exceptions	
oop	
reports	
research	
services	
storage	
system	
users	

OVERVIEW

PACKAGE

TREE

INDEX

SEARCH

HELP

controllers

Contents

Description

Classes and Interfaces

Package controllers

package controllers

Classes

Class	Description
AdminController	
AttendanceController	
Controller	
CourseController	
EmployeeController	
MarkController	
NewsController	
ResearchController	
StudentController	
TeacherController	
UserCreationController	

OVERVIEWCLASSTREEINDEXSEARCHHELP

controllers > AdminController

Search documentation

Contents

Filter contents (type .)

Description

Field Summary

Constructor Summary

Method Summary

Constructor Details

AdminController(UDBM)

Method Details

addUser(User, String)

addNews(Scanner, Active)

removeUser(String)

printAllUsers()

createManager(Scanner)

createAdmin(Scanner)

createFullTimeResearcher(Scanner)

createStudent(Scanner)

createTeacher(Scanner)

assignTeacherToCourse(Scanner)

void addNews(Scanner scanner, Active user)

void addUser(User user, String password)

void assignHeadLector(Scanner scanner)

void assignTeacherToCourse(Scanner scanner)

void createAdmin(Scanner scanner)

void createFullTimeResearcher(Scanner scanner)

void createLesson(Scanner scanner)

void createManager(Scanner scanner)

void createStudent(Scanner scanner)

void createTeacher(Scanner scanner)

void printAllEmployees()

void printAllStudent()

void printAllTeachers()

void printAllUsers()

void registerStudentToCourse(Scanner scanner)

void removeUser(String id)

Methods inherited from class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Hierarchy For All Packages

Package Hierarchies:

academic, comparators, controllers, database, demo, enums, exceptions, oop, reports, research, services, storage, system, users

Class Hierarchy

- java.lang.Object[Ⓜ]
 - system.Active (implements java.io.Serializable[Ⓜ])
 - system.Admin
 - controllers.AttendanceController
 - services.AttendanceService
 - system.Authenticate
 - controllers.Controller
 - controllers.AdminController
 - controllers.CourseController
 - controllers.EmployeeController
 - controllers.MarkController
 - controllers.NewsController
 - controllers.ResearchController
 - controllers.StudentController
 - controllers.TeacherController
 - controllers.UserCreationController
 - academic.Course (implements java.io.Serializable[Ⓜ])
 - services.CourseService
 - storage.DataStorage
 - services.EmployeeService
 - services.FindService
 - academic.Lesson (implements java.io.Serializable[Ⓜ])
 - demo.Main
 - oop.Main
 - storage.ManualDataStorage
 - storage.ManualDataStorage
 - academic.Mark (implements java.io.Serializable[Ⓜ])
 - services.MarkService
 - system.News
 - services.NewsService
 - system.OperationalManager
 - comparators.PaperByCitationsComparator (implements java.util.Comparator[Ⓜ]<T>)
 - comparators.PaperByDateComparator (implements java.util.Comparator[Ⓜ]<T>)
 - comparators.PaperByPagesComparator (implements java.util.Comparator[Ⓜ]<T>)
 - services.ReadService
 - reports.Report (implements java.io.Serializable[Ⓜ])
 - research.ResearchPaper (implements java.lang.Comparable[Ⓜ]<T>, java.io.Serializable[Ⓜ])
 - research.ResearchProject (implements java.io.Serializable[Ⓜ])
 - services.ResearchService
 - services.StudentService
 - java.lang.Throwable[Ⓜ] (implements java.io.Serializable[Ⓜ])
 - java.lang.Exception[Ⓜ]
 - exceptions.AuthenticationException
 - exceptions.ContentNotFoundException
 - exceptions.CreditLimitException
 - exceptions.LowGPAException
 - exceptions.LowHIndexException
 - exceptions.NotResearcherException
 - exceptions.UserNotFoundException
 - database.UDBM
 - system.User
 - users.User (implements java.io.Serializable[Ⓜ])
 - users.Employee
 - users.Admin
 - users.FullTimeResearcher (implements research.Researcher)
 - users.Manager
 - users.Teacher (implements research.Researcher)
 - users.Student (implements research.Researcher)
-
- database.UDBM
 - system.User
 - users.User (implements java.io.Serializable[Ⓜ])
 - users.Employee
 - users.Admin
 - users.FullTimeResearcher (implements research.Researcher)
 - users.Manager
 - users.Teacher (implements research.Researcher)
 - users.Student (implements research.Researcher)
 - services.UserService

Interface Hierarchy

- research.Researcher

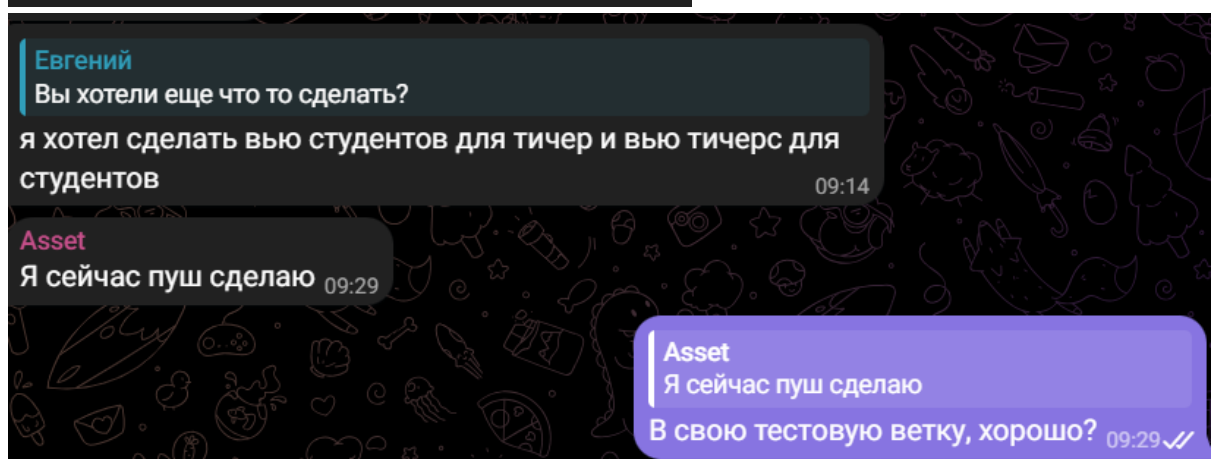
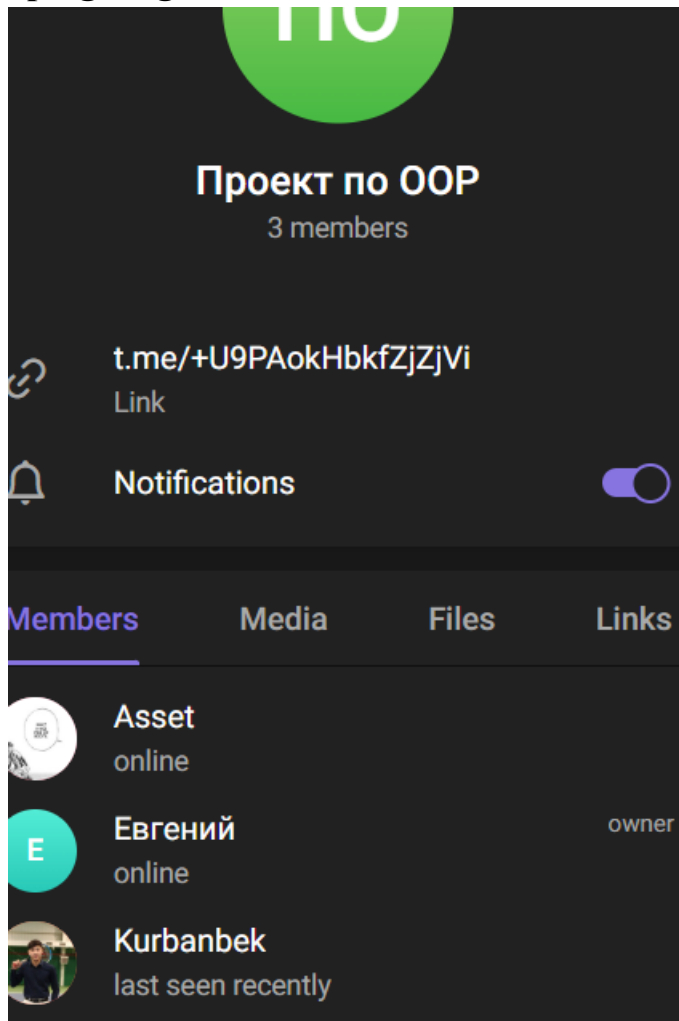
Enum Class Hierarchy

- java.lang.Object[Ⓜ]
 - java.lang.Enum[Ⓜ]<E> (implements java.lang.Comparable[Ⓜ]<T>, java.lang.constant.Constable[Ⓜ], java.io.Serializable[Ⓜ])
 - enums.AttendanceStatus
 - enums.DepartmentType
 - enums.LessonType
 - enums.ManagerType
 - enums.TeacherRank
 - enums.UserRole
 - system.UserType

And so on, there are a lot of docs and we can't insert them all into the report

Communication:

We established our own telegram group just as team was assembled and started to discuss our view of the project. Work in the group was a great thing and we used our best sides to create this project. Although, due to different schedule among our team members, the team work has been asynchronous from each other and had a "ping"/lag.



Problems and Obstacles we encountered so far or left in the code:

1. Lack of similar scheduling: We have different schedules across the team, so we couldn't work at the same time, thus our team work had a "ping"/lag
2. Our initial code wasn't prepare for scaling things, so we had to rewrite it almost fully.
3. We struggled a lot with database functions, because we haven't thought of using Spring framework
4. We have lack of transactions with our database, so there can be bugs and errors when multiple users operate at the same time(not our case, but possibility still stands)
5. Our current relying on constructors (heavy one) is restricting our ability to scale our basic classes up, such as Student and etc. We would like to improve it, but the deadline is near and that would require full refactor of UDBM code to use setters and separate map/serialization classes(we haven't made them enough).
6. Our team vision on the project wasn't unified for some time, so we often were creating duplicated or "strange" code.